Introduction to Software Engineering

Requirements Analysis

The student team is required to complete the Software Requirements Specification (SRS) document for the assigned course project, following the attached template.

Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

Objectives 1

1 Member Contribution Assessment 2

2 Problem Statement 3

3 Requirements Overview 4

4 Requirements Analysis 5

5 Prototype/Mockup 6

Software Requirements Specification

Objectives

This document focus on the following topics:

- ✔ Complete the Software Requirements Specification (SRS) document with the following contents:
  - Elaborate on the Problem Statement
  - Overview of Requirements (Functional and Non-Functional), Stakeholders
  - Use Case Model
  - Use Case Specifications
  - Create Prototype and Mockup Diagrams of the System Interface
- ✔ Đọc hiểu tài liệu phân tích yêu cầu.

1 Member Contribution Assessment

ID	Name	Role	Contribution (%)	Signature
23127061	Trương Lý Khải	**Project Manager** Business Analyst Tester	100%	
23127207	Đặng Đăng Khoa	Designer Backend Developer	100%	
23127271	Võ Ngọc Bích Trâm	Designer Frontend Developer Tester	100%	
23127404	Lê Tuấn Lộc	Fullstack Developer & Engineer	100%	
23127486	Phan Quốc Thịnh	Designer Frontend Developer	100%	

2 Problem Statement

When playing any Pokémon main series games (e.g., Pokémon Black, Pokémon Sword), **players** aiming to finish the whole game often need to **find specific Pokémons** to complete their Pokédex. They must currently search through multiple, ad-filled, and often slow-to-load wikis or complicated and overbearing guides just to find which route or area the Pokémon appears in. Our team's goal is to create a single, fast, and simple **web application** to allow players to **build a community and search for Pokémons** effectively and efficiently.

For the operating environment, the website will be implemented using ReactJS and Tailwind CSS for the user interface, NodeJS with the Express framework for controlling website logic and player data. Hosting the website is done through the local server, supporting HTML5. The main programming language throughout the project is JavaScript.

Upon launching the website, players will **log in** to their accounts and **customize** their username and password to become users. On the main page, users can **view others' posts and follow other users** to see their posts more frequently. Users can also **discuss Pokémons and their games** under any posts through a like feature and a comment system. Posts can be customized to allow for both text and images.

Along with users, there will also be **moderators** employed by the main website's hosts. Moderators can **warn other users about their posts or delete them** if they feel that the posts are inappropriate. Moderators are differentiated from users through special highlighted tags on their profiles. If a post is deleted or a warning is issued, the user will be informed the next time they load the website.

Along with the dashboard, players can also access the map. When using the map, players can select their desired Pokémon game. By default, the website will **display the total map** of the most recent release of the Pokémon IP (Pokémon Z-A as of 2025). There is also a **game selector** for the player, should they want to select another game to track.

Each map can be **zoomed in or zoomed out** depending on the selected location. A map will also display the name of each specific region of the map that is named in the game using layered text. A detailed list of every region in a game is shown below the game selector, and players can scroll and **select a specific region** from there. Choosing a region will move the current view of the map to the center of the region.

Near the region list will be a table showing **every type of Pokémon available** within the game. Pokémons are categorized either by type (Grass, Electric, ...) or by rarity (Rare, Legendary,...). There will also be a search bar for the players to **search Pokémon names**. Players can **tag any Pokémon that they want to track**. The map will display all possible locations that a tagged Pokémon can appear, using a pin containing the corresponding Pokémon's icon. For each pin, if a player has explored and exhausted the corresponding area, they can **mark the pin** (once the area can spawn Pokémons again, the player can unmark marked pins). Hovering over each pin will also **display the method of engaging with the Pokémon** in that area. The location of each Pokémon is fetched from the public PokéAPI.

Game and region selection, selected Pokémons, tagged pins, and current map state are data stored specifically to each user for the map while usernames, passwords, profiles, posts, comments, and likes history are stored using MongoDB.

3 Requirements Overview

3.1 Stakeholders

STT	Stakeholder	Description
1	Project Sponsor	Responsible for the majority of the funding towards the project.
2	Players/Users	The main stakeholders who will be using the main features of the software.
3	Moderators	Moderators work under the Project Sponsor and the Project Team to maintain the social environment within the software.
4	Pokémon Parent Company	The project operates under the usage of their IP and intellectual work (art, design, mechanics).

3.2 Requirements

3.2.1. Functional Requirements Specification

3.2.1.A. User Requirements - Main Page

- When the User accesses the website for the first time, they must gain access to the login window.

- When the User inputs their login information, the website must return the result to the user.

- The User should be able to choose to register an account while using the login window.

- The User must see the avatar clearly upon seeing the main page.

- The User should be able to access the profile customization page through the avatar.

- The website must show a list of posts on the main page for the User to interact with.

- Interaction with posts should include liking and commenting on a post.

- When the User likes a post, the website must return the updated count.

- When the User comments on a post, the website should update the page to allow the user to type in their comment.

- The User should be able to see the avatar of the original poster of each post.

- The User should be able to see the profile page of another User through the avatar.

- If a User is warned for inappropriate behaviour or has their post deleted, the message must be dominant on the screen in bright colors and the reasoning must be given in a different font to signify its importance.

- The User should be able to access the map directly from the main page.

3.2.1.B. Moderator-Specific Requirements - Main Page

- When a Moderator finds an inappropriate post, selecting the original poster's avatar must also show the option to warn the user or delete the post.

- When a Moderator chooses to warn the user or delete the post, a prompt must be given to the Moderator to fill in the reasoning in text.

3.2.1.C. Player Requirements - Map Page

- The Player should be able to access the main page from the map.

- The Player should be able to select a Pokémon from a list given by the system.

- The Player should be able to use the map along with an easy-to-use zoom function.

- When the Player requires tracking a Pokémon, all of its relevant pins must be shown on the map.

3.2.2. Non-Functional Requirements Specification

3.2.2.A. Performance Requirements

- The website must load the main page within 3 seconds under normal network conditions.

- All user interactions (liking, commenting, loading posts) must provide visual feedback within 5 seconds.

- The map interface must render completely within 5 seconds after loading.

- Pin marking/unmarking operations must update the map display within 2 seconds.

3.2.2.B. Usability Requirements

- All interactive elements must have an adequate size.

- The zoom function on the map must be controllable through both pinch gestures and explicit zoom buttons.

- On the profile page, the logout button should be visually clear and distinguishable from everything else.

- The option to warn the user or delete the post must be visually different from the usual User's selection to signify its importance.

- When the Player marks or unmarks a pin, the difference between the color contrast of both pins must be significant.

3.2.2.C. Reliability Requirements

- The system must maintain uptime from 6 AM to 10 PM.

- User data must be automatically saved after every important action (likes, comments, follows).

- Error rates for post creations must be lower than 1%.

3.2.2.D. Security Requirements

- All user credentials must be encrypted using a hashing algorithm (bcrypt).

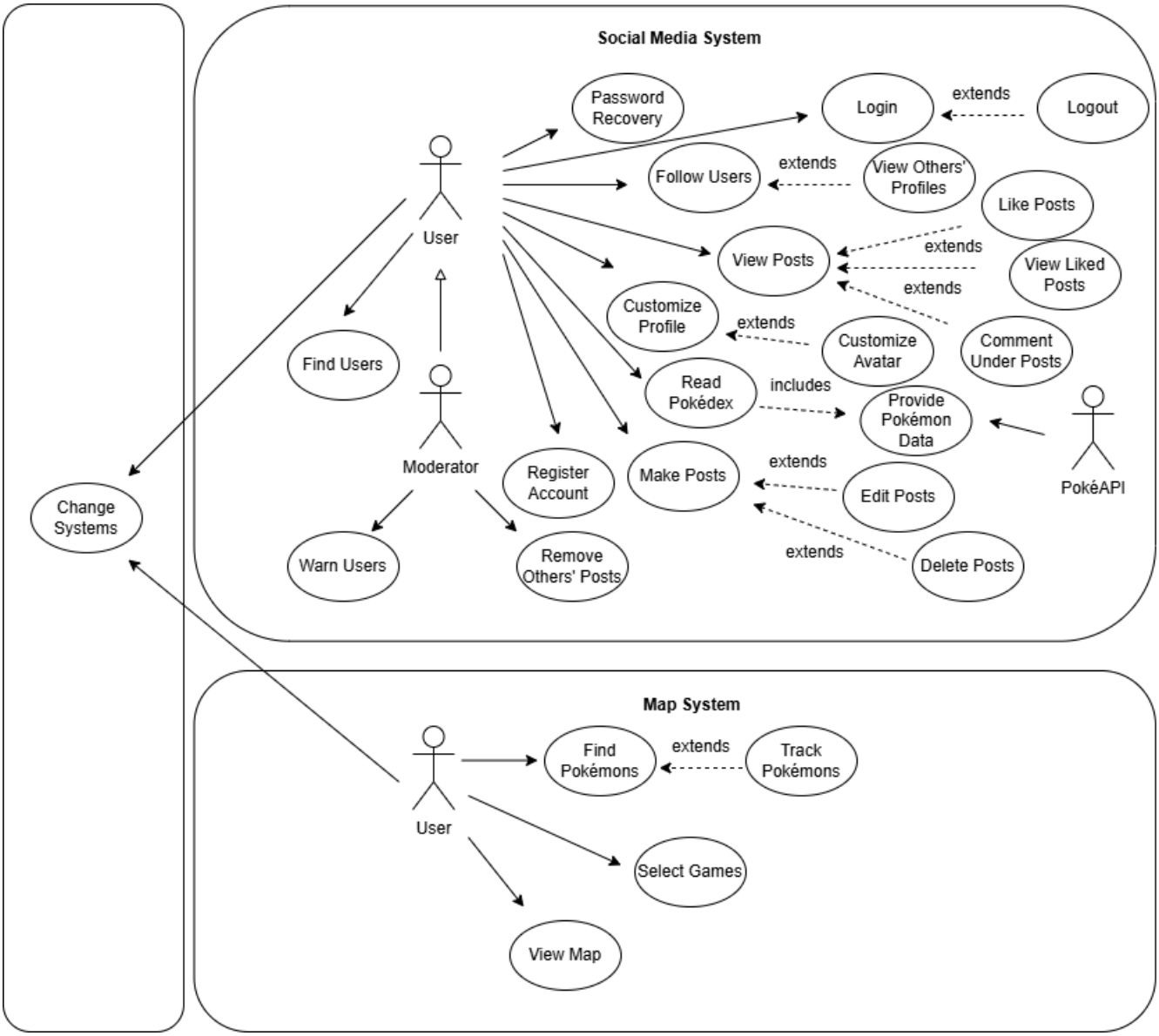
- Moderator actions (warnings, post deletions) must be logged in and have an additional safeguard.

- One-Time Passwords (OTP) for registration must have a strict validity period of 10 minutes.

- The system must limit OTP resend requests to a maximum of 3 attempts per email/phone number per hour.

4 Requirements Analysis

4.1 *Use Case model*



4.2 Use Case Specification

4.2.1. Use Case 1

Use case ID	U001
Use Case	**Register Accounts**
Brief Description	This feature is accounted for when a user creates their first account or an alternate account to use on the main page.
Actor	User(Player), Account System.
Pre-Condition	The user must be at the login form.
Result	The account's data is created and stored within the database.
Main Scenario	1. User presses the button to register for a new account. 2. System displays a form for the User to input. 3. User types in their username and password and their email address. 4. The system creates an account and stores the account's data in the database. 5. System validates the input format and sends a One-Time Password (OTP) to the provided email/phone. 6. User enters the received OTP into the verification field. 7. System verifies the OTP code. 8. System updates the interface to the main page.
Alternative Scenarios	4a. System detects that an account with the same username already exists. 5a. System returns to the register form while displaying an error about the existing account. 4b. System detects that the username, password, or email/phone is invalid. 5b. System returns to the register form while displaying an error about the invalid input. 6c. User enters an incorrect or expired OTP. 7c. System displays an OTP verification error and allows the user to request a new OTP. 6d. OTP is not delivered within the timeout period. 7d. System allows the user to resend a new OTP code.
Non-Functional Constraints	The username/email check must complete within 3 seconds. OTP delivery must occur within 30 seconds. OTP validity period must be 10 minutes. The system must support OTP retry every 60 seconds.

4.2.2. Use case 2

Use case ID	U002
Use Case	**Login**
Brief Description	This feature is accounted for when a user logs into the website.
Actor	User(Player), Account System
Pre-Condition	The user must be at the login form.
Result	The user is logged in to the system and gains access to the main webpage.
Main Scenario	1. User enters their registered username and password. 2. User presses the login button. 3. System verifies the credentials against the database. 4. System authenticates the user and loads their account data. 5. System updates the interface to the main page.
Alternative Scenarios	3a. System detects that the username does not exist. 4a. System returns to the login form while displaying an error to the User about invalid credentials. 3b. System detects that the password is incorrect. 4b. System returns to the login form while displaying an error to the User about invalid credentials.
Non-Functional Constraints	The credential verification process must be completed within 3 seconds. Error messages must be displayed for a minimum of 5 seconds.

4.2.3. Use case 3

Use case ID	U003
Use Case	**Logout**
Brief Description	This feature is accounted for when a user logs out of the system in any method.
Actor	User(Player), Account System
Pre-Condition	The user is logged in to the system and has access to the social media webpage.
Result	The user has logged out successfully and can select to log in to another account.

Main Scenario	1. User hovers/holds a press on their avatar. 2. System displays an option under their avatar to log out. 3. User presses the button to log out. 4. System adds a safeguard for the user to confirm if they want to log out. 5. User confirms their choice. 6. System stores the current data of the user and leads the webpage back to the login UI.
Alternative Scenarios	1a. User exits the website. 2a. Continue from step 6 of the Main Flow. 5b. User does not confirm their choice. 6b. System reverts the website back to its original state.
Non-Functional Constraints	The logout process must take 1 second.

4.2.4. Use case 4

Use case ID	U004
Use Case	**Follow Users**
Brief Description	This feature is accounted for when a user selects 'Follow' other users.
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in to the system and has access to the social media webpage.
Result	User A follows user B successfully.
Main Scenario	1. User A presses the "Follow" button on other user B's profile. 2. The backend system stores a new record showing that user A follows user B in the database. 3. After storing successfully, the system updates the UI to reflect the new status.
Alternative Scenarios	3a. Connection or Server error occurs while the system is processing to store the record. 4a. The system notifies that the request has failed. 5a. User A may try again.
Non-Functional Constraints	The system must show whether the following request is successful or not within 2 seconds. After following successfully, the "Follow" button displayed on the UI is changed into "Following", and the "Followers" number of user B is updated. A failed follow request results in a pop-up section with the content "Follow Request Failed. Try again" being displayed.

4.2.5. Use Case 5

Use case ID	U005
Use Case	**View Others' Profiles**
Brief Description	This feature is accounted for when a user sees information about another user's profile page.
Actor	Users(Player), Social Media System.
Pre-Condition	The user has access to the social media webpage.
Result	The user gains information about another user.
Main Scenario	1. User A presses the avatar of another user B. 2. System displays the profile page of user B.
Alternative Scenarios	1a. User A hovers/holds a press on the avatar of another user B. 2a. System displays a small section of user B's profile under the avatar.
Non-Functional Constraints	After 2 seconds of hovering/holding, the profile section must appear. The system must show the profile page of the selected user in 3 seconds. The profile page of the selected user must be in the same format of the editing section of the current user.

4.2.6. Use case 6

Use case ID	U006
Use Case	**View Posts**
Brief Description	This feature is accounted for when a user accesses the social media dashboard.
Actor	User(Player), Social Media System
Pre-Condition	The user has access to the social media webpage.
Result	Display posts from other users on the dashboard.
Main Scenario	1. User gains access to the social media dashboard. 2. The system connects to the database to get information about posts related to the current user. 3. After fetching data successfully, posts are displayed on the dashboard.

Alternative Scenarios	3a. Connection or Server error occurs while the system is fetching the data 4a. The system notifies that it has failed to load posts. 5a. User refreshes the page. 3b. No posts available for the current user, the system retrieves empty data. 4b. The system notifies that there is no available posts yet.
Non-Functional Constraints	Each post must have the avatar and name of the original poster along with the time they posted. System validates data within 5 seconds. When errors occur, a pop-up section is displayed to notify and asks the user to refresh the page. When there are no available posts, notify the user with a text with the content "No posts to display".

4.2.7. Use case 7

Use case ID	U007
Use Case	**Comment Under Posts**
Brief Description	This feature is accounted for when a user wants to comment under a post.
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in to the system and viewing posts on the social media dashboard.
Result	User leaves a comment under a post successfully.
Main Scenario	1. User chooses the comment section under the post. 2. A text area is displayed for the user to type their comment. 3. User types a comment and presses the “Submit” button. 4. The system stores comment information in the database. 5. Comment is successfully uploaded.
Alternative Scenarios	5a. Connection or Server error occurs while the system is storing the data 6a. The system notifies that it has failed to comment. 7a. The user may submit again.
Non-Functional Constraints	The notification indicating whether the comment is successfully uploaded or not must be shown within 5 seconds. On any post that the user has commented on, their comments will always be shown first.

4.2.8. Use case 8

Use case ID	U008
Use Case	**Like Posts**
Brief Description	This feature is accounted for when a user wants to like a post.
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in to the system and viewing posts on the social media dashboard.
Result	User leaves a “like” to the post successfully.
Main Scenario	1. User clicks the “Like” button displayed under the post. 2. The system stores this information in the database. 3. After storing successfully, the system updates the UI to reflect the new status.
Alternative Scenarios	3a. Connection or Server error occurs while the system is storing the data 4a. The system notifies that it has failed to leave a like.
Non-Functional Constraints	Upon liking a post, the likes count display must update within 5 seconds, along with a color indicator that the user has liked the post.

4.2.9. Use case 9

Use case ID	U009
Use Case	**Customize Profile**
Brief Description	This feature is accounted for when a user wants to edit his/her own profile.
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in to the system and then goes to his or her profile page.
Result	The user’s profile is changed based on related modifications.
Main Scenario	1. User chooses the “Edit profile” option to start editing. 2. The system shows an editing section allowing the user to modify the Avatar, Name, and Tagline. 3. User clicks “Save” to confirm the changes. 4. The system saves the changes to the profile to the database. 5. The system informs that changes are saved successfully.

Alternative Scenarios	5a. Connection or Server error occurs while the system is storing the data. 6a. The system notifies that it has failed to modify the profile. 3b. The user clicks "Cancel" instead of "Save". 4b. The system discards all changes and returns to the profile page.
Non-Functional Constraints	The system must support common image formats: PNG and JPEG. Image size must not exceed 5MB. Name field must not exceed 50 characters. The tagline field must not exceed 250 characters. The system must validate the data within 5 seconds.

4.2.10. Use case 10

Use case ID	U010
Use Case	**Make Posts**
Brief Description	This feature is accounted for when a user wants to upload a post.
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in to the dashboard.
Result	A new post from the user is uploaded and is visible to other users.
Main Scenario	1. User clicks on the “Create Post” button. 2. The system shows a section allowing the user to put their content and relevant images. 3. User clicks “Upload” to confirm the post. 4. The system saves the post’s data to the database. 5. The new post is successfully uploaded.
Alternative Scenarios	5a. Connection or Server error occurs while the system is storing the data. 6a. The system notifies that it has failed to upload the post. 3b. The user clicks "Cancel" instead of "Upload". 4b. The system discards all changes and returns to the dashboard.
Non-Functional Constraints	The system must support common image formats: PNG, JPEG. Related images size must not exceed 5MB. The button to create a new post must be in a fixed location, above the posts list.

4.2.11. Use case 11

Use case ID	U011
Use Case	**Edit Posts**
Brief Description	This feature is accounted for when a user edits a post upon creation or edits an old post.
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in and is viewing one of their own existing posts.
Result	The post's data is updated in the database, and the changes are displayed.

Main Scenario	1. User clicks the “Edit” button on one of their own posts. 2. The system opens the post in an editable form, pre-filled with the current content and images. 3. User modifies the text and/or replaces the images. 4. User clicks the “Save” button to confirm the edits. 5. The system validates the new data and updates the post in the database. 6. The system closes the edit form and displays the updated post.
Alternative Scenarios	5a. Connection or Server error occurs while the system is storing the data. 6a. The system notifies that it has failed to edit the post. 5b. The system detects that the post content is blank after editing. 6b. The system displays an error message stating the post cannot be empty and returns to the editable form. 4c. The user clicks "Cancel" instead of "Save" at any point during step 3 or 4. 5c. The system discards all changes and returns to displaying the original post.
Non-Functional Constraints	The "Edit" button must only be visible on a user's own posts. The transition to and from the edit mode must be seamless, taking less than 1 second.

4.2.12. Use case 12

Use case ID	U012
Use Case	**Delete Posts**
Brief Description	This feature is accounted for when a user deletes one of their posts.
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in and is viewing one of their own existing posts.
Result	The post is permanently removed from the system and is no longer visible to any user.
Main Scenario	1. User clicks the “Delete” button on one of their own posts. 2. The system displays a confirmation dialog asking the user to confirm the deletion. 3. User confirms the deletion. 4. The system removes the post's data and any associated images from the database. 5. The system removes the post from the user's profile and

	the main feed. 6. The system displays a success message confirming the post has been deleted.
Alternative Scenarios	5a. Connection or Server error occurs while the system is querying the data. 6a. The system notifies that it has failed to delete the post. 3b. The user cancels the deletion in the confirmation dialog. 4b. The system closes the dialog and no changes are made to the post.
Non-Functional Constraints	The "Delete" button must only be visible on a user's own posts. The deletion operation must be completed within 3 seconds. A deleted post cannot be recovered by regular users.

4.2.13. Use case 13

Use case ID	U013
Use Case	**Find Users**
Brief Description	This feature is accounted for when a user uses the search bar to search for the username of another user
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in on the main webpage.
Result	The system displays a list of user profiles that match the search query.
Main Scenario	1. User navigates to the search bar. 2. User types in a username or part of a username. 3. While typing, the system dynamically queries the database for matching usernames. 4. The system displays a list of relevant user profiles (username and profile picture) in a dropdown. 5. User clicks on a profile from the list.
Alternative Scenarios	4a. The system finds no matching usernames. 5a. The system displays a "No users found" message in the dropdown. 3b. The user submits the search query. 4b. The system navigates to a dedicated search results page showing a full list of all matching user profiles.
Non-Functional Constraints	The search results in the dropdown must update with a delay of no more than 5 seconds after the user stops typing.

4.2.14. Use case 14

Use case ID	U014
Use Case	**Read Pokédex**
Brief Description	This feature is accounted for when a user uses the Pokédex.
Actor	User(Player), Pokédex System
Pre-Condition	The user is logged in on the main webpage.
Result	The user successfully gets the information of an entry in the Pokédex.
Main Scenario	1. User selects the Pokédex icon. 2. System requests and receives Pokédex information from the PokéAPI 3. System opens up the Pokédex and shows a list of entries in order of ascending entry numbers. 4. User selects the entry they want. 5. The system displays the full information of that entry on the screen.
Alternative Scenarios	4a. User searches for the entry number through the search bar. 5a. The system displays the full information of the entry with the corresponding number on the screen. 4b. User searches for the Pokémon name through the search bar. 5a. The system displays the full information of the entry with the corresponding Pokémon on the screen.
Non-Functional Constraints	The Pokédex must load within 10 seconds. The Pokédex search function must be done within 5 seconds. Each entry must be organized and compact as a list of uniform rows.

4.2.15. Use case 15

Use case ID	U015
Use Case	**Provide Pokémon Data**
Brief Description	This feature is accounted for when the PokéAPI provides data to the system.
Actor	PokéAPI, Pokédex System

Pre-Condition	PokéAPI must establish a stable connection to the Pokédex System
Result	Pokémon Data is successfully transferred to the Pokédex.
Main Scenario	1. System establishes a connection to the PokéAPI. 2. System requests Pokédex information from the PokéAPI. 3. PokéAPI searches and fetches the requested list of data. 4. PokéAPI transfers the data to the system. 5. System confirms that data has been received.
Alternative Scenarios	4a. PokéAPI returns a "404 Not Found" error code and an error message. 5a. System displays an error to the user stating the Pokémon data was not found. 2b. The connection between the System and the PokéAPI is lost during the request. 3b. The System's request times out. 4b. The System displays a connection error to the user and allows them to retry.
Non-Functional Constraints	The API response time for a successful data fetch must be under 2 seconds. The system must cache successfully fetched Pokémon data to reduce API calls. The data transfer must use the secure HTTPS protocol to ensure data integrity.

4.2.16. Use case 16

Use case ID	U016
Use Case	**Warn Other Users**
Brief Description	This feature is accounted for when a moderator detects a user who does not behave appropriately online and chooses to warn them.
Actor	Moderator, User(Player) - passive, Social Media System
Pre-Condition	The moderator is logged into the moderation panel and is viewing the profile of the user in question.
Result	A formal warning is issued to the user and recorded in the system.
Main Scenario	1. The moderator selects the option to "Warn User" from the user's profile or a reported post. 2. The system presents the moderator with a form to fill in the warning reason. 3. The moderator fills in the reasoning for the warning. 4. System adds for full confirmation from the moderator.

	5. Moderator confirms the warning. 6. System records the warning in the user's moderation history. 7. The system automatically sends a notification to the warned user, detailing the reason for the warning. 8. The warned user will receive the notification the next time they log in.
Alternative Scenarios	5a. The moderator cancels the warning action. 6a. The system closes the form without issuing a warning or making any changes.
Non-Functional Constraints	The warning notification must be delivered to the user within 30 seconds of the moderator's confirmation. All warning actions must be permanently logged and associated with the moderator's account ID.

4.2.17. Use case 17

Use case ID	U017
Use Case	**Remove Others' Posts**
Brief Description	This feature is accounted for when a moderator detects an inappropriate post and decides to remove it.
Actor	Moderator, User(Player) - passive, Social Media System
Pre-Condition	The moderator is logged into the moderation panel and is viewing the inappropriate post.
Result	The post is removed from public view.
Main Scenario	1. The moderator clicks the button associated with removing the post. 2. The system presents a form requiring the moderator to fill in the reason for removal. 3. The moderator fills in a reason and confirms the removal. 4. System removes the post from public view as well as the post's data within the database. 5. System sends a notification to the original poster that their post was removed and provides the reason. 6. The user will receive a notification of the removed post with a reason the next time they log in.

Alternative Scenarios	3a. The moderator cancels the action in the dialog. 4a. The system closes the dialog, and the post remains visible. 5b. The post has already been removed by another moderator or the user. 6b. The system displays an error message to the moderator stating, "This post has already been removed or no longer exists."
Non-Functional Constraints	The removal action must be processed and reflected across the system within 1 minute.

4.2.18. Use case 18

Use case ID	U018
Use Case	**Select Game**
Brief Description	This feature is accounted for when a user uses the map and wishes to select a game to view.
Actor	User(Player), Map System
Pre-Condition	The user must be using the interactive map.
Result	The desired game's map and Pokémon list are shown..
Main Scenario	1. User clicks on the "Select Game" dropdown menu on the map interface. 2. System displays a list of available games. 3. User selects a specific game from the list. 4. System loads the corresponding regional map and the list of Pokémon available in that game. 5. System updates the interface to display the new map and the associated Pokémon list.
Alternative Scenarios	3a. The user clicks/taps outside the dropdown menu without making a selection (this counts as an accident). 4a. The system closes the dropdown menu, and the current map view remains unchanged. 4b. The system encounters an error while loading the new map data. 5b. The system displays an error message ("Failed to load map data") and reverts to the previously selected map.
Non-Functional Constraints	The transition between different regional maps must be seamless, with a load time of under 2 seconds.

4.2.19. Use case 19

Use case ID	U019
Use Case	**View Map**
Brief Description	This feature is accounted for when a user uses the functionalities of the interactive map (zoom, pan).
Actor	User(Player), Map System
Pre-Condition	The user must be using the interactive map.
Result	The desired game's map and Pokémon list is shown.
Main Scenario	1. User performs a navigation action either through direct interaction or through keys. 2. The system captures the input and calculates the new map viewport. 3. The system renders the updated section of the map. 4. The user can continue to interact with the map, repeating steps 1-3
Alternative Scenarios	2a. The user attempts to pan beyond the map's boundaries. 3a. The system prevents movement after a custom threshold and reverts the change.. 2b. The user zooms in/out to a limit of the zoom level. 3b. The system disables further zoom functionality.
Non-Functional Constraints	The map should respond to user input with a latency of less than a second.

4.2.20. Use case 20

Use case ID	U020
Use Case	**Find Pokémons**
Brief Description	This feature is accounted for when a user uses the map to search for Pokémons in any method.
Actor	User(Player), Map System
Pre-Condition	The user must be using the interactive map and must have already selected a game.
Result	The desired Pokémon is selected and shown on the map.
Main Scenario	1. System displays a complete list of all Pokémons on the sidebar.

	2. User chooses the icon of the Pokémon they want to search. 3. System displays the location of all viable locations of that specific Pokémon on the map as pins.
Alternative Scenarios	2a. User searches for the name of the Pokémon. 3a. System rearranges the list to show Pokémons that match closest to the specified name. 4a. Continue from step 2 of the main flow. 2b. User searches for the type of Pokémon. 3b. System rearranges the list to only show Pokémons that match the specified type. 4b. Continue from step 2 of the main flow.
Non-Functional Constraints	The name search must take 5 seconds to complete. The type search must take 5 seconds to complete. The pins shown on the map must be adjustable and must scale visually to the map size when zoomed in or out.

4.2.21. Use case 21

Use case ID	U021
Use Case	**Track Pokémons**
Brief Description	This feature is accounted for when a user uses the map to track the location of Pokémons.
Actor	User(Player), Map System
Pre-Condition	The user must be using the interactive map, must have already selected a game, and must have found the Pokémon to track.
Result	The user successfully tracks and records the progress of the locations of the chosen Pokémon.
Main Scenario	1. System displays the location of all viable locations of the chosen specific Pokémon on the map as pins. 2. User focuses on one of the unmarked pins. 3. User explores the corresponding game at the pin's location and finds the Pokémon. 4. User marks the pin. 5. System stores the pin's data as marked and visually fades the pin. 6. User continues from step 2 or finishes tracking the Pokémon
Alternative Scenarios	6b. User restarts the tracking process by unmarking all marked pins of the selected Pokémon.

Non-Functional Constraints	Both marking and unmarking a pin must take 2 seconds to update the pin on the map.

4.2.22. Use case 22

Use case ID	U022
Use Case	**Change Systems**
Brief Description	This feature is accounted for when a user switches from the interactive map to the main webpage and vice versa.
Actor	User(Player), Map System, Social Media System
Pre-Condition	The user is logged in and is actively using one of the two systems (either the Map System or the Social Media System).
Result	The user interface successfully transitions to the target system, and the user's session in the previous system is preserved as data.
Main Scenario	1. User clicks on the navigation element for the other system. 2. The current system saves the user's state. 3. The current system loads the interface and initial state of the target system. 4. The target system restores the user's relevant state. 5. The target system's interface is displayed to the user.
Alternative Scenarios	3a. The user has an unsaved post draft in the Social Media System. 4a. The system displays a confirmation dialog warning the user that their draft will be lost if they proceed. 5a. If the user confirms, the draft is discarded, and the navigation continues. If the user cancels, the navigation is aborted. 4b. The target system fails to load due to a network error. 5b. The system displays an error message and allows the user to retry the navigation or return to the previous system.
Non-Functional Constraints	The transition between systems must be completed in less than 3 seconds. The navigation must have a loading indicator displayed during the transition. User authentication must be shared between systems, requiring no re-login during the switch.

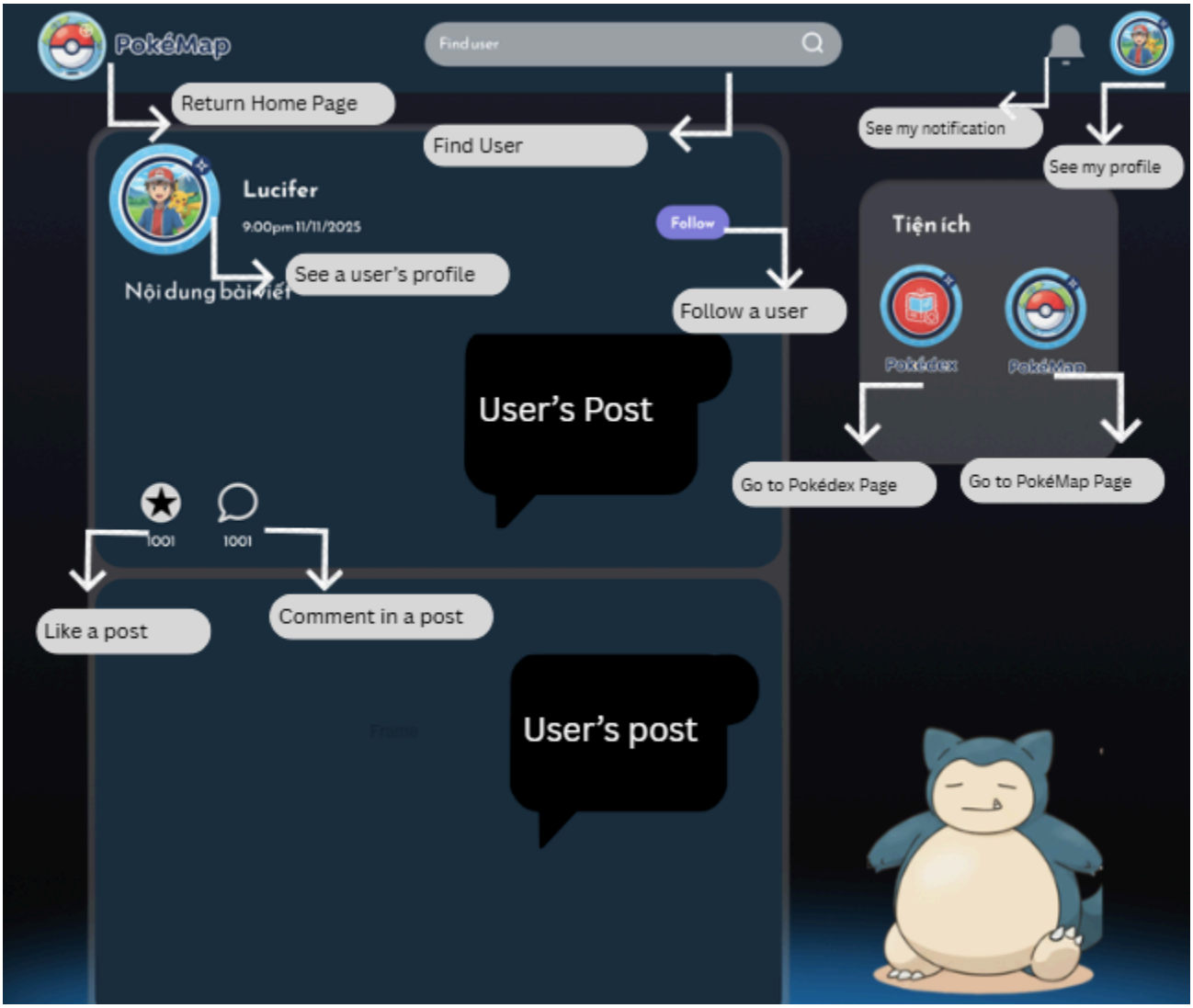
4.2.23. Use case 23

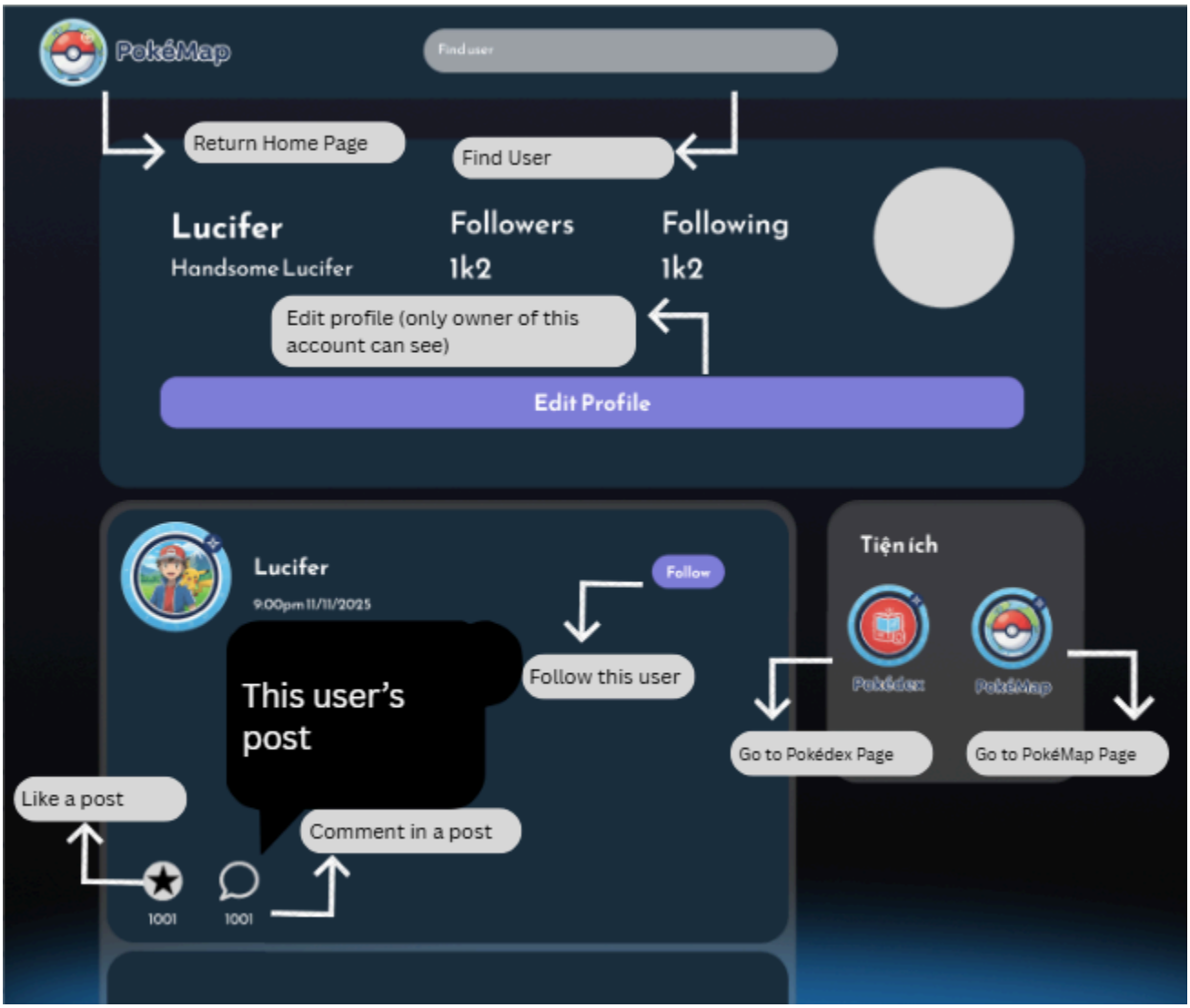
Use case ID	U022
Use Case	**Password Recovery**
Brief Description	This feature is accounted for when the user wishes to change their password or recover their password using the email.
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in and is actively viewing their profile.
Result	The user's password is successfully changed, and a confirmation is sent to their registered email.
Main Scenario	1. User navigates to their profile settings and selects the "Change Password" option. 2. System prompts the user to enter their current password for verification. 3. User enters their current password correctly. 4. System prompts the user to enter and confirm a new password. 5. User enters and confirms the new password. 6. System validates the new password meets security requirements. 7. System updates the password in the database and invalidates existing sessions. 8. System sends a confirmation email to the user's registered email address. 9. System displays a success message confirming the password change.
Alternative Scenarios	3a. The user has an unsaved post draft in the Social Media System. 4a. The system displays a confirmation dialog warning the user that their draft will be lost if they proceed. 5a. If the user confirms, the draft is discarded, and the navigation continues. If the user cancels, the navigation is aborted. 4b. The target system fails to load due to a network error. 5b. The system displays an error message and allows the user to retry the navigation or return to the previous system.
Non-Functional Constraints	The transition between systems must be completed in less than 3 seconds. The navigation must have a loading indicator displayed during the transition. User authentication must be shared between systems, requiring no re-login during the switch.

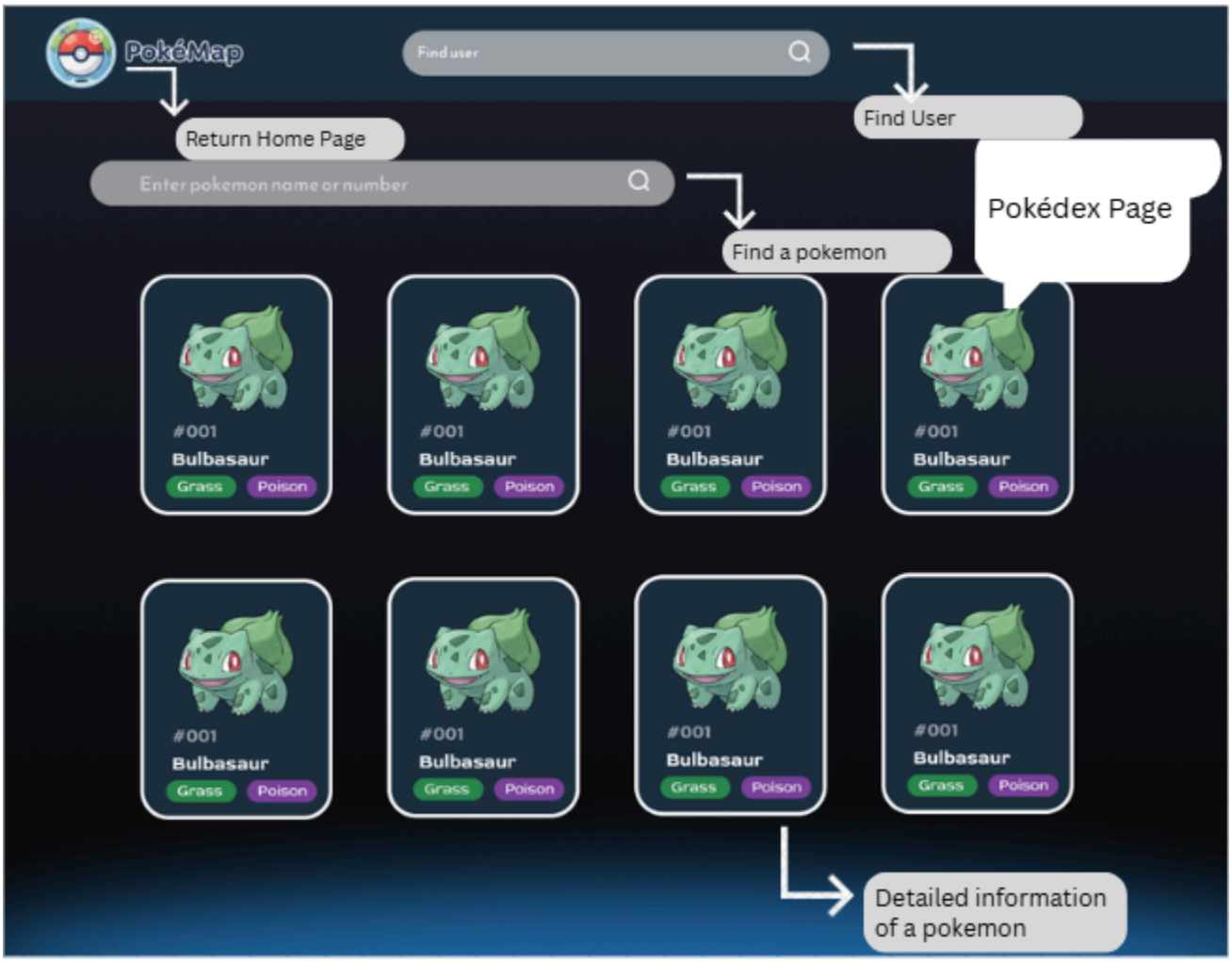
4.2.24. Use case 23

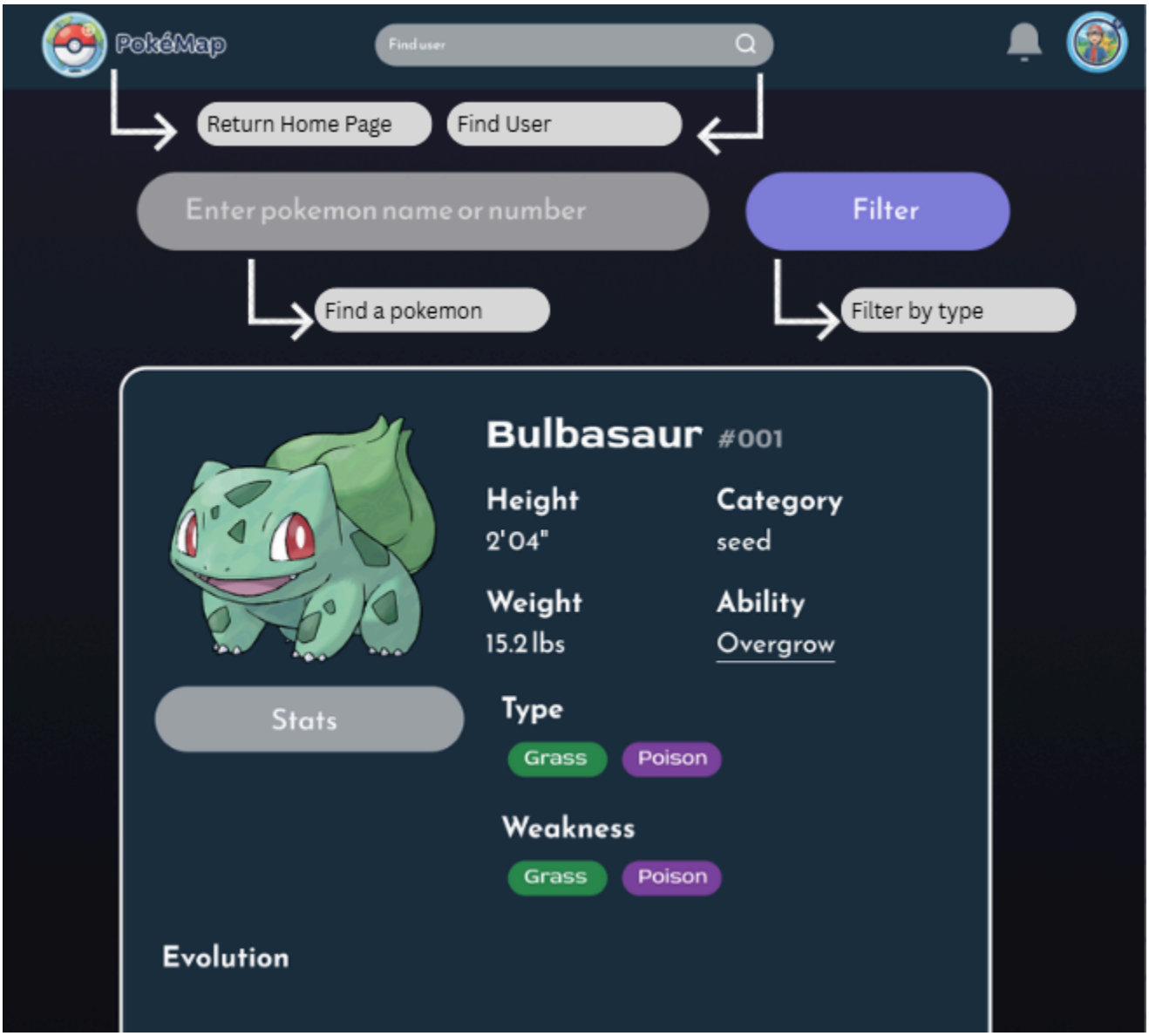
Use case ID	U023
Use Case	**View Liked Posts**
Brief Description	This feature is accounted for when a user filters out the dashboard so that they can only see posts that they've liked.
Actor	User(Player), Social Media System
Pre-Condition	The user is logged in and is on the main dashboard/post feed.
Result	The dashboard feed is filtered to display only the posts the user has previously liked.
Main Scenario	1. User selects the option on the dashboard to filter the "Liked Posts" from the menu. 2. System queries the database for all posts the current user has liked. 3. System generates a filtered feed containing only these posts, sorted chronologically (newest first). 4. System updates the dashboard interface to display the "Liked Posts" feed.
Alternative Scenarios	2a. The user has not liked any posts. 3a. The system displays a message, such as "You haven't liked any posts yet," and shows an empty feed. 5b. The user clicks the "Clear Filter" or "Show All" button. 6b. The system reloads the standard, unfiltered dashboard feed.
Non-Functional Constraints	The filtered feed of liked posts must load within 2 seconds of the user's selection.

5 Prototype/Mockup




PokéMap
Find user
Return Home Page
Find User
Lucifer
Handsome Lucifer
Followers
1k2
Following
1k2
Edit profile (only owner of this account can see)
Edit Profile
Lucifer
9:00pm 11/11/2025
Follow
Follow this user
This user's post
Like a post
Comment in a post
1001
1001
Tiện ích
Pokédex
PokéMap
Go to Pokédex Page
Go to PokéMap Page


PokéMap
Find user
Find User
Return Home Page
Enter pokemon name or number
Pokédex Page
Find a pokemon
#001
Bulbasaur
Grass
Poison
Detailed information of a pokemon


PokéMap
Find user
Return Home Page
Find User
Enter pokemon name or number
Filter
Find a pokemon
Filter by type
Bulbasaur #001
Height
2'04"
Category
seed
Weight
15.2 lbs
Ability
Overgrow
Stats
Type
Grass
Poison
Weakness
Grass
Poison
Evolution